Second Chance Technical Report

Albert Sun, Danny Zheng, Isaac Thomas, Nabil Chowdhury

https://www.secondchancesupport.me/

Purpose

The purpose of this project is to help people in Texas who were previously convicted of a crime reintegrate into society. We provide statistics to bring awareness to this community, as well as provide lists of re-entry programs and rehabilitation facilities that can be located by proximity.

User Stories

Phase I:

1. Re-entry: Add contact information (phone number or email) for each reentry facility
    a. We planned to do this initially so we already worked on it. We added phone numbers to each facility for this phase. We plan to automate this by scraping data off our data source for this model.
2. Rehab: Add a link to their website for each rehab center
    a. This was also something we planned to do. We have each rehab center's website linked below on its card below the rest of its attributes. We plan to automate this by scraping data off our data source for this model.
3. Re-entry: Add a link to the website for the reentry facility if available
    a. This was also something we planned to do. We have each re-entry center's website linked below on its card below the rest of its attributes. We plan to automate this by scraping data off our data source for this model.
4. Rehab: Allow rehab centers to be filterable/sortable by rating (for the first phase just add an attribute to each rehab instance with a corresponding rating)
    a. Since we are not implementing filtering or sorting in this phase, we only accomplished what was stated for the first phase. We added ratings to each of our rehab centers taken from Google Maps and plan to automate this using the Google Maps API.
5. County: Add the local jail/prison that each county feeds into as part of each county instance.
    a. We accomplished this by putting the main jail/prison where the majority of the inmates are located by searching it up online. We plan to automate this for more instances by finding a data source that provides the population of jails/prisons in different counties or the whole state.

Phase II:

1. Re-entry: Fix the Re-entry page, the re-entry page seems to just display a pixelated image.
    a. We mistakenly set a photo's property to fill its container, which ended up being the whole page. This was a simple fix, as we just removed the layout property of the image element.
2. Home Page Layout: There seem to be some odd layout choices and quite a bit of space on the home page. Make the video thumbnail on the bottom larger.
    a. We achieved this by moving the features of the home page around to a more favorable layout. We also stretched the video thumbnail to match the width of the explore cards above it.
3. About Page Layout: Add icons/links for Tools used on the About page or at least a bigger clickable element that links to that tool/data source.
    a. We added cards with each name of a tool used, a photo with the tool's logo, and the ability for the card to be clicked, which leads to the tool's website.
4. Instance Card Layout: Would like to see uniformity on the instance cards (mainly looking at rehab cards) so that the text below the titles is on the same level
    a. We added some functionality from Flexbox to make the text under the title uniform.
5. Search bar and filterable dropdown: Add a search bar and filterable dropdown for model pages
    a. We added the UI components for a search bar and filter dropdown, but we didn't implement the search and filter functionality, as this is what is focused on in Phase 3.

Phase III:

1. County Instances: The Instances for the counties don't seem to be loading. I'd also like the model cards to be a little better styled, as it looks pretty rudimentary.
    a. We didn't update something in our docker file for the backend so we were getting "error loading." We fixed this issue. We will refine the styling of the model cards next phase.
2. Sortable Items: I don't see any sorting functionality for the model pages. Please implement this.
    a. We implemented this with an interactive search, sort, and filter UI component.
3. Filterable Items: Please fix the filterable items in each model page to work and be applicable to each page.
    a. We implemented this with an interactive search, sort, and filter UI component.
4. Rehab Instances: When I click on instances in the Rehab model page, it fails to load. Please fix.
    a. We changed out the reentry table and added and updated new fields for the SQL Alchemy models on the backend server code interfacing with the database. Fixed.

5. Reentry Model Page: When I navigate to re-entry, nothing loads (Error Loading). Please fix.
    a. We changed out the reentry table and added and updated new fields for the SQL Alchemy models on the backend server code interfacing with the database. Fixed.

Models

Re-entry Programs: Re-entry programs give people the support they need to successfully return to their communities after incarceration and can greatly improve public safety.

Attributes:

- City
- County
- Name
- Hours/Days of Operation
- Type of office
- Website

Rehab Centers: Rehab centers are intensive, supervised programs designed to help people stop using alcohol and other drugs and give them the tools they need to live a healthy life.

Attributes:

- Name
- City
- County
- Facility type
- Type of payment accepted
- Website

Counties: Counties are political and administrative divisions of a state, providing certain local governmental services.

- Name
- Number of convicts
- Number of ex-convicts (released)
- Population
- Types of Conviction

- Website

Front-end Architecture

Our front end is made using Next.js with TypeScript, Tailwind CSS, and components from Shadcn.ui.

Related Documentation:

Next.js: https://nextjs.org/docs
TypeScript: https://www.typescriptlang.org/docs/
Tailwind: https://tailwindcss.com/docs/installation
Shadcnui: https://ui.shadcn.com/docs

File System:

Our frontend is structured as follows:

/app - all pages in our application
/app/about - directory for about page
/app/[model] (county, re-entry, rehab) - model pages
/app/[model]/[id] - instance pages for each model

/components - created components and
/components/ui - components imported from Shadcn

/lib - a collection of utils

How to add new pages:

To add a page, create directories corresponding with the route name. For example, create /app/about/hello directory for a page at /about/hello. Then create a page.tsx file in the directory. To create a dynamic route, put brackets around the directory name, such as /about/hello/[id].

Navbar and Footer:

The navbar and footer are located in the components directory. They are inserted into every page in the layout.tsx located at /app/layout.tsx

Model Cards:

Re-entry Schema:

- name: name of re-entry program
- address_1: main address, such as the street name and house number
- address_2: additional information such as apartment numbers, suite numbers, or other details that can help in identifying the exact location of the address.
- city: the city program is located in
- state_abbr: abbreviation of the state the program is located in
- state_name: the state the program is located in
- zip: zip code associated with the location of the program
- phone: phone number to reach the program
- program_type: the specification of the program
- open_hour: the hours of operation
- center_is_open: whether the program is open
- general_email: email address used to reach the program
- latitude: geographical latitude
- longitude: geographical longitude
- website_url: URL to get to the program's website
- fax: fax information
- b64_img: image of program in b64 format

Rehab Schema:

- name: name of rehab center
- website: URL link to center website
- address: address of center
- services_0: service offered at the center
- services_1: service offered at the center
- services_2: service offered at the center
- payment_0: payment type accepted
- payment_1: payment type accepted
- payment_2: payment type accepted
- payment_3: payment type accepted
- payment_4: payment type accepted
- payment_5: payment type accepted
- type: specification of rehab at center
- phone: phone number to reach the center
- b64_img: image of center in b64 format
- services_3: service offered at the center
- payment_6: payment type accepted
- payment_7: payment type accepted

- payment_8: payment type accepted
- latitude: geographical latitude
- longitude: geographical longitude

County Schema:

- name: name of county
- population: population of the county
- population_rank: rank of county in Texas counties based on population
- type_violent: number of convicts for violent charges in county
- type_drug: number of convicts for drug charges in county
- type_property: number of convicts for property charges in county
- type_other: number of convicts for other charges in county
- type_total: total number of convicts in the county
- b64_img: image of center in b64 format
- latitude: geographical latitude
- longitude: geographical longitude

Back-end Architecture

API Documentation:

6 API endpoints

1. secondchancesupport.me/api/v1/rehabs
    a. Grabs all rehab centers
2. secondchancesupport.me/api/v1/rehabs?page=int
    a. Grabs instances starting from page (set to be 20 instances per page).
3. secondchancesupport.me/api/v1/reentry?id=int
    a. Grabs reentry programs from the backend based on a unique ID for each reentry program.
4. secondchancesupport.me/api/v1/reentries
    a. Grabs all reentry programs from the backend.
5. secondchancesupport.me/api/v1/rehabs?page=int
    a. Grabs instances starting from page (set to be 20 instances per page).
6. secondchancesupport.me/api/v1/rehab?id=int
    a. Grab a rehab center based on a unique ID for each rehab center.
7. secondchancesupport.me/api/v1/county?id=int
    a. Grab a county, based on unique IDs for each county.
8. secondchancesupport.me/api/v1/counties
    a. Grabs all counties with their associated prisoner statistics and released prisoner statistics.
9. secondchancesupport.me/api/v1/rehabs?page=int
    a. Grabs instances starting from page (set to be 20 instances per page).

Parsing Download Data

Phase I

- For Phase I, to obtain prisoner statistics for each county, we first downloaded the data from here https://data.texas.gov/dataset/Texas-Department-of-Criminal-Justice-Releases-FY-2/htfi-jkdg/about_data and then click on the export button in the top right corner below the sign in button. Download with the CSV option and then move this file into the same folder as prisonerParse.py, located in backend/parsers/prisonerParse.py. Then, run python prisonerParse.py, and it should output a prisoner.json file.
- The current format of the prisoner.json file will be later changed to utilize an array, instead of having each county be a distinct key, but for now, the format is each county is a key, with a type property containing five different properties. Each represents the type of crime they committed. Each property has a value representing the number of prisoners that were released on these charges from 2020 to 2019 in this county.
- The rest of the data is manually gotten from the websites. For reentry programs, the data was manually entered from https://www.careeronestop.org/Developers/WebAPI/ReEntryPrograms/list-reentry-program-contacts.aspx. For rehab centers, the data is entered from https://findtreatment.gov/locator, typing in Texas into the search bar, and checking the state checkbox.

Phase II

- In Phase II, we created a database with Supabase. We entered the data into the database by converting the JSON data files for counties, rehab centers, and re-entry resources to SQL. We ran these queries on a tool called TablePlus, which can connect to Supabase and run large queries. This was very helpful because Supabase doesn't let you run large queries on the free plan so you would have to split the file into many small files.
- We set up the backend server using…Flask for the endpoints, SQLAlchemy for the ORM, PostgreSQL with Supabase as the hosting provider.
- We web-scrapped images for the instances using:
    - Selenium
    - Beautiful Soup
    - Requests
- Images for the County are actually img links from wikipedia. They're displayed as <img src={img_url}>. These img_url are stored in Supabase.
- Images are stored in Supabase, as b64_img. The string is encoded in base64 encoding, and further encoded as ASCII. This resulting string is stored in the database. To display the image on frontend, do <img src={"" + instance.b64_img}></img>
- We created endpoints for our API to:
    - Grab All Counties

- Grab All Reentries
- Grab All Rehab Centers
- Grab One Rehab Center by ID
- Grab One Reentry Center by ID
- Grab One County by ID

Phase III

- In Phase III, we implemented an api for filtering, searching, and sorting on the backend and implemented a corresponding ui that calls this backend api. We also added new columns to the SQL database and web scrapped more information for the models.
- Example of backend api call
  - cs373-backend.secondchancesupport.me/api/v1/search?model=rehab&page=1
- All Calls to Search endpoints follow this format of search?model=<replace with model type>. Further url parameters will start with &. All queries support &page=<page num> (replace <page num> with desired actual page number). Doing &page returns a page, or 20 instances, to the user in json form. In the form below
- {
- “Page”: <pagenumber>,
- “Results”: [<Array of instances>],
- “total”:<total instances>,
- “totalPages”: <Math.ceil(number of instances / 20)>
- }
- Parameters For New Search Endpoint
  - Model
    - Rehab. Rehab Specific Parameters below.
      - The Fields Below Are Searchable on the Frontend, e.g. user input is arbitrary.
      - Query: Gives instances with information containing the user inputted words. Example call: &query=hi,you will return all instances with the words, “hi” and “you”. This system also works with numbers, e.g. &query=4.
        - The Fields Below are Filterable, e.g. the options are preset, not arbitrary.
      - Type: The preset values can be found in frontend > app > rehab > page.tsx treatmentTypes array.
        - Fields below takes in a comma separated list of values, and finds instances containing a substring of those values. Ex: &payments=Medicare,State (values not real). These values are filterable, e.g. they have preset values. These preset values can be found in frontend > app > rehab > paymentOptions.tsx.
      - Payments
      - Services

        - This is a Boolean True or False, On Frontend, Checking this sorts any searchable fields by descending. If unchecked, makes searchable fields ascending.
    - Sort:
        - Asc or Desc, sort the searchables (e.g. name and address1 for reentry model will be sorted alphabetically).
    -
    - Limit (Not Useful)
- County. County specific parameters below.
        - The Field Below Are Searchable on the Frontend, e.g. user input is arbitrary.
    - Query: Gives instances with information containing the user inputted words. Example call: &query=hi,you will return all instances with the words, “hi” and “you”. This system also works with numbers, e.g. &query=4.
        - All Parameters Below are Range Filters. Example: &population=1,2. Where 1 is the lower and 2 is the upper, finds all instances with fields in between lower and upper range, inclusive.
    - Typeviolent
    - Typeproperty
    - Typeother
    - Typedrug
    - Typetotal
    - Population_rank
    - Population
- Reentry. Reentry specific parameters below.
        - The Fields Below are Filterable, e.g. the options are preset, not arbitrary.
    - Programtype: Predefined Options defined in frontend folder, under filterOptions.tsx under frontend > app > re-entry > filterOptions.tsx, and programTypeOptions.
    - County: Predefined Options defined in frontend folder, under filterOptions.tsx under frontend > app > re-entry > filterOptions.tsx, and countyOptions.
        - The Fields Below Are Searchable on the Frontend, e.g. user input is arbitrary.
    - Query: Gives instances with information containing the user inputted words. Example call: &query=hi,you will return all instances with the words, “hi” and “you”. This system also works with numbers, e.g. &query=4.
        - The Fields Below is a Range Filter, takes in two comma separated values, 1st value is lower range, 2nd value is upper range, inclusive.

                - Rating: Float, 0 means there's no rating at all. Ex: &rating=1.2,3.
                    - This is a Boolean True or False, On Frontend, Checking this sorts any searchable fields by descending. If unchecked, makes searchable fields ascending.
                - Sort:
                    - Asc or Desc, sort the searchables (e.g. name and address1 for reentry model will be sorted alphabetically).
        - Page
            - Any page number.
- Database Changes.
- Cumulated_payments, and cumulated_services. These 2 columns take payments and services and concatenate the strings together. Used on backend for filtering by payments and services easier (only need to find the substring with the payment in cumulated_payments).
- Added Ratings and County / Zip Code data for Reentry and Rehab. Data web scrapped from Google Maps.

To Get started with backend, do python endpoints.py (make sure to have keys.env in same folder as endpoints.py, keys.env should contain DB_PASSWORD=<Replace with your databases's remote password> and DB_USERNAME=<Replace with your databases's remote username>).

To Get started with frontend, cd into frontend, and do npm install, npm run dev.

Database

We created a PostgreSQL database hosted on Supabase. This database is populated by tables for counties, re-entry programs, rehab centers, and geographical info for these models.

Hosting:

- AWS Amplify
- AWS EC2
- Supabase
- Namecheap

Data Sources:

- List ReEntry Program Contacts | Web API | CareerOneStop
- Search For Treatment - FindTreatment.gov
- Texas Department of Criminal Justice Releases FY 2020 | Open Data Portal (For county statistics)
- https://www.google.com/maps/place/ (For Images of Rehab and Reentry centers)
- https://www.zip-codes.com/state/tx.asp (For Zip to County and County to Zip)
- https://www.texas-demographics.com/counties_by_population (For population)

- https://en.wikipedia.org/wiki/ (For county images, grabs wikimedia image link)

Testing

- Unit tests
- Postman API Tests
- Acceptance Tests
    - Uses Selenium, UnitTest and Pytest
    - Run ‘pytest main.py’ in \frontend\tests\acceptance_tests

Tools

Hosting/Logistics:

- AWS Amplify: hosts our full-stack website
- NameCheap: provides a domain name to host our website to
- Gitlab: hosts our project repository, issue tracking, and pipelines
- Postman: used to document our API
- Discord: main form of communication
- Slack: used to communicate with TA
- Python: Parsing raw data into JSON form
- AWS EC2: For hosting backend API

Front-end:

- Next.js: https://nextjs.org/docs: For client-side routing and React for creating UI.
- TypeScript: https://www.typescriptlang.org/docs/: For frontend scripting, to give type safety.
- Tailwind: https://tailwindcss.com/docs/installation: CSS Framework for responsive and clean styling of React components.
- Shadcnui: https://ui.shadcn.com/docs: Set of ready-made React components for easy UI creation and design.
- Jest: Frontend testing.
- Axios: Making API calls.

Back-end:

- Python: Parsing data from CSV to JSON.
- Flask: Creating API endpoints.
- SQLAlchemy: ORM for interfacing with remote databases hosted by Supabase.
- Postgresql: Database dialect being used.
- Selenium: Web scraping and collecting images. Also for acceptance testing for the GUI.

- Supabase: Database to store data for instances
- TablePlus: Running SQL Queries to update the database

Challenges

Phase I:

Initially, we took a while to get rolling with the project. We couldn't figure out how to set up our domain name off of Namecheap for AWS. Our biggest challenge was time management. We struggled to make time to get the project completed due to time conflicts.

Some of the previous data sources we planned on using were insufficient for the number of instances that we needed. Thus, we had to research and find new data sources.

We overcome these challenges by maintaining good communication, reaching out to our TA, and discussing potential replacements for our data sources adequately.

Phase II:

We had a lot of trouble setting up our backend server through the tutorials but we eventually figured out some supplemental info to help.

We ran into a lot of problems with web scraping images. It was difficult to find image sources for the sheer amount of instances we had. Some instances weren't getting images at all and some instances were getting completely random photos, like sushi. We had to research better ways to web scrape.

We overcome these challenges by maintaining good communication, reaching out to our TA, and discussing potential fixes to issues we ran through together

Phase III:

- Originally, the payments and services url parameters were dependent on the order. To make payments and services order agnostic, the cumulated_payments and cumulated_services columns were constructed. Then, looping over the list of payments and services given by the url parameters and checking if the payment or service is a substring of cumulated_payments or cumulated_services made the problem solvable.
- The backend server was not updating with new backend code, so the entire process of creating the backend server and running it had to be recreated.
- Dealing with the sheer number of url parameters for the search endpoint was quite difficult to be maintainable, extracting the logic and using a for loop made it much more maintainable and scalable.